

■ LEVEL 24 • DFS

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Explore a connected region or branch completely. Use recursion or an explicit stack and mark visited nodes.
Reference Examples	Number of Islands • Flood Fill • Clone Graph • Path Existence • Cycle Detection • Connected Components
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 24 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Number of Islands	DFS / Connected Components	#200
2	Flood Fill	DFS / Grid	#733
3	Clone Graph	DFS + HashMap	#133
4	Path Sum III	Tree DFS + Prefix Sum	#437
5	All Paths From Source Lead to Destination	DFS + Cycle Detection	#1059
6	Connected Components	DFS / Graph Traversal	Absent
7	Number of Islands	DFS Grid	200
8	Flood Fill	DFS	733
9	Path Sum	Tree DFS	112
10	Clone Graph	DFS + HashMap	133

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Number of Islands

Pattern: DFS / Connected Components

LeetCode: #200

■ Input	2D grid
■ Output	number of islands
■ Constraints	land is '1', water is '0'
■ Main Idea / Pattern	Start DFS from every unvisited land cell and mark the entire component.
■■ Time Complexity	O(mn)
■ Space Complexity	O(mn)

■ Optimal C++ Code

```
void dfs(int r, int c, vector<vector<char>>& g) {
    if (r < 0 || r >= g.size() ||
        c < 0 || c >= g[0].size() ||
        g[r][c] != '1')
        return;

    g[r][c] = '0';

    dfs(r + 1, c, g);
    dfs(r - 1, c, g);
    dfs(r, c + 1, g);
    dfs(r, c - 1, g);
}

int numIslands(vector<vector<char>>& grid) {
    int ans = 0;

    for (int r = 0; r < grid.size(); r++)
        for (int c = 0; c < grid[0].size(); c++)
            if (grid[r][c] == '1') {
                ans++;
                dfs(r, c, grid);
            }

    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Flood Fill

Pattern: DFS / Grid

LeetCode: #733

■ Input	image grid, starting cell, new color
■ Output	image after flood fill
■ Constraints	four-direction connectivity
■ Main Idea / Pattern	DFS through connected cells having the original color, changing them as visited.
■■ Time Complexity	O(mn)
■ Space Complexity	O(mn)

■ Optimal C++ Code

```
void dfs(int r, int c, int old, int color,
        vector<vector<int>>& image) {
    if (r < 0 || r >= image.size() ||
        c < 0 || c >= image[0].size() ||
        image[r][c] != old)
        return;

    image[r][c] = color;

    dfs(r + 1, c, old, color, image);
    dfs(r - 1, c, old, color, image);
    dfs(r, c + 1, old, color, image);
    dfs(r, c - 1, old, color, image);
}

vector<vector<int>> floodFill(vector<vector<int>>& image,
                             int sr, int sc, int color) {
    if (image[sr][sc] == color) return image;

    int old = image[sr][sc];
    dfs(sr, sc, old, color, image);
    return image;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Clone Graph

Pattern: DFS + HashMap

LeetCode: #133

■ Input	connected graph node
■ Output	deep cloned graph
■ Constraints	node labels unique
■ Main Idea / Pattern	Cache clones before exploring neighbors so cycles do not recurse forever.
■■ Time Complexity	$O(V+E)$
■ Space Complexity	$O(V)$

■ Optimal C++ Code

```
unordered_map<Node*, Node*> mp;

Node* dfs(Node* node) {
    if (!node) return nullptr;
    if (mp.count(node)) return mp[node];

    Node* copy = new Node(node->val);
    mp[node] = copy;

    for (Node* nei : node->neighbors)
        copy->neighbors.push_back(dfs(nei));

    return copy;
}

Node* cloneGraph(Node* node) {
    return dfs(node);
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Path Sum III

Pattern: Tree DFS + Prefix Sum

LeetCode: #437

■ Input	binary tree and target sum
■ Output	number of downward paths summing to target
■ Constraints	path can start/end anywhere but moves downward
■ Main Idea / Pattern	Use prefix sums along the current root-to-node path. Count earlier prefix = current-target.
■ Time Complexity	O(n)
■ Space Complexity	O(h)

■ Optimal C++ Code

```
int dfs(TreeNode* root, long long cur,
        long long target, unordered_map<long long, int>& freq) {
    if (!root) return 0;
    cur += root->val;
    int ans = freq.count(cur - target) ? freq[cur - target] : 0;
    freq[cur]++;
    ans += dfs(root->left, cur, target, freq);
    ans += dfs(root->right, cur, target, freq);
    freq[cur]--;
    return ans;
}

int pathSum(TreeNode* root, int targetSum) {
    unordered_map<long long, int> freq;
    freq[0] = 1;
    return dfs(root, 0, targetSum, freq);
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 All Paths From Source Lead to Destination

Pattern: DFS + Cycle Detection

LeetCode: #1059

■ Input	directed graph and source/destination
■ Output	whether every path ends at destination
■ Constraints	destination has no outgoing edge
■ Main Idea / Pattern	DFS each reachable node with three states: unvisited, visiting, done. A cycle or wrong terminal invalidates the condition.
■ Time Complexity	$O(V+E)$
■ Space Complexity	$O(V)$

■ Optimal C++ Code

```
bool dfs(int u, int dest, vector<vector<int>>& g,
        vector<int>& state) {
    if (g[u].empty())
        return u == dest;

    if (state[u] == 1) return false;
    if (state[u] == 2) return true;

    state[u] = 1;
    for (int v : g[u])
        if (!dfs(v, dest, g, state))
            return false;

    state[u] = 2;
    return true;
}

bool leadsToDestination(int n, vector<vector<int>>& edges,
                        int source, int destination) {
    vector<vector<int>> g(n);
    for (auto& e : edges)
        g[e[0]].push_back(e[1]);

    vector<int> state(n);
    return dfs(source, destination, g, state);
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Connected Components

Pattern: DFS / Graph Traversal

LeetCode: ■ Absent

■ Input	n nodes and undirected edges
■ Output	number of connected components
■ Constraints	nodes labeled 0..n-1
■ Main Idea / Pattern	Build an adjacency list and start DFS from every unvisited node.
■■ Time Complexity	$O(V+E)$
■ Space Complexity	$O(V+E)$

■ Optimal C++ Code

```
void dfs(int u, vector<vector<int>>& g, vector<bool>& vis) {
    vis[u] = true;
    for (int v : g[u])
        if (!vis[v])
            dfs(v, g, vis);
}

int countComponents(int n, vector<vector<int>>& edges) {
    vector<vector<int>> g(n);
    for (auto& e : edges) {
        g[e[0]].push_back(e[1]);
        g[e[1]].push_back(e[0]);
    }
    vector<bool> vis(n);
    int components = 0;
    for (int i = 0; i < n; i++) {
        if (!vis[i]) {
            components++;
            dfs(i, g, vis);
        }
    }
    return components;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Number of Islands

Pattern: DFS Grid

LeetCode: 200

Input	grid
Output	number of connected land components
Constraints	rows, cols <= 300
Main Idea / Pattern	DFS from each unvisited land cell and mark its entire connected component.
Time Complexity	O(mn)
Space Complexity	O(mn)

Optimal C++ Code

```
class Solution {
    int m, n;

    void dfs(vector<vector<char>>& grid, int r, int c) {
        if (r < 0 || r >= m || c < 0 || c >= n ||
            grid[r][c] != '1') return;

        grid[r][c] = '0';

        dfs(grid, r + 1, c);
        dfs(grid, r - 1, c);
        dfs(grid, r, c + 1);
        dfs(grid, r, c - 1);
    }

public:
    int numIslands(vector<vector<char>>& grid) {
        m = grid.size();
        n = grid[0].size();
        int ans = 0;

        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                if (grid[r][c] == '1') {
                    ans++;
                    dfs(grid, r, c);
                }
            }
        }

        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Flood Fill

Pattern: DFS

LeetCode: 733

Input	image, sr, sc, color
Output	image after flood fill
Constraints	rows, cols <= 50
Main Idea / Pattern	Replace the starting component using DFS, stopping at boundaries and different colors.
Time Complexity	O(mn)
Space Complexity	O(mn)

Optimal C++ Code

```
class Solution {
    int m, n;
    int oldColor, newColor;
    void dfs(vector<vector<int>>& image, int r, int c) {
        if (r < 0 || r >= m || c < 0 || c >= n ||
            image[r][c] != oldColor) return;
        image[r][c] = newColor;
        dfs(image, r + 1, c);
        dfs(image, r - 1, c);
        dfs(image, r, c + 1);
        dfs(image, r, c - 1);
    }
public:
    vector<vector<int>> floodFill(vector<vector<int>>& image,
                                int sr, int sc, int color) {
        m = image.size();
        n = image[0].size();
        oldColor = image[sr][sc];
        newColor = color;
        if (oldColor != newColor) {
            dfs(image, sr, sc);
        }
        return image;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Path Sum

Pattern: Tree DFS

LeetCode: 112

Input	root, targetSum
Output	whether a root-to-leaf path has target sum
Constraints	0 <= nodes <= 5000
Main Idea / Pattern	Subtract each node value from the target and check whether a leaf reaches zero.
Time Complexity	O(n)
Space Complexity	O(h)

Optimal C++ Code

```
class Solution {
public:
    bool hasPathSum(TreeNode* root, int targetSum) {
        if (!root) return false;
        if (!root->left && !root->right) {
            return root->val == targetSum;
        }
        return hasPathSum(root->left, targetSum - root->val) ||
            hasPathSum(root->right, targetSum - root->val);
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Clone Graph

Pattern: DFS + HashMap

LeetCode: 133

Input	node
Output	deep copy of a connected graph
Constraints	0 <= nodes <= 100
Main Idea / Pattern	Clone each node once and recursively clone its neighbors using a map from original to copy.
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

Optimal C++ Code

```
class Solution {
    unordered_map<Node*, Node*> mp;
public:
    Node* cloneGraph(Node* node) {
        if (!node) return nullptr;
        if (mp.count(node)) return mp[node];
        Node* copy = new Node(node->val);
        mp[node] = copy;
        for (Node* nei : node->neighbors) {
            copy->neighbors.push_back(cloneGraph(nei));
        }
        return copy;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.